

Asymptotic Analysis

- A way to compare speed of two possible solutions to the same problem.
- Basic idea is to think of the rate of growth of running time of an algorithm.
 - How much slower does the algorithm get if we double the input size?
 - we focus on big inputs.

For asymptotic analysis, we will first define what is the rate of growth of a function.

(Big Oh and its related definitions)

Big O

Def: Let $f, g: \mathbb{R}^{\geq 0} \rightarrow \mathbb{R}^{\geq 0}$, we

say that f grows no faster

than g if there exist a constant

$c > 0$ and $n_0 \geq 0$ s.t

$$\forall n \geq n_0 : f(n) \leq c \cdot g(n)$$

In this case, we write

$f(n)$ is $O(g(n))$ or

$$f(n) = O(g(n))$$

Examples $f(n) = 3n^2 + 2$ is $O(n^2)$

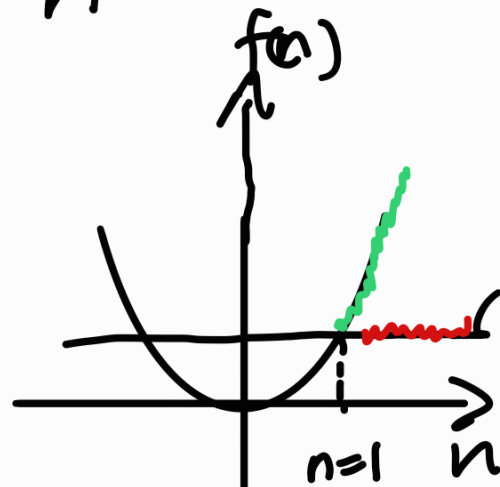
If $f(n) = O(n^2)$, then

$$[\exists c > 0, n_0 \geq 0 : \forall n \geq n_0 : f(n) \leq c \cdot g(n)]$$

WTS

$$3n^2 + 2 \leq c \cdot n^2$$

$$\forall n \geq 1: 2 \leq 2n^2$$



$$\forall n \geq 1: 3n^2 + 2 \leq 3n^2 + 2n^2$$

$$\forall n \geq 1: \underbrace{3n^2 + 2}_{f(n)} \leq \underbrace{5n^2}_{g(n)}$$

we have $c=5$, $n_0=1$

$$\forall n \geq n_0 : f(n) \leq c g(n)$$

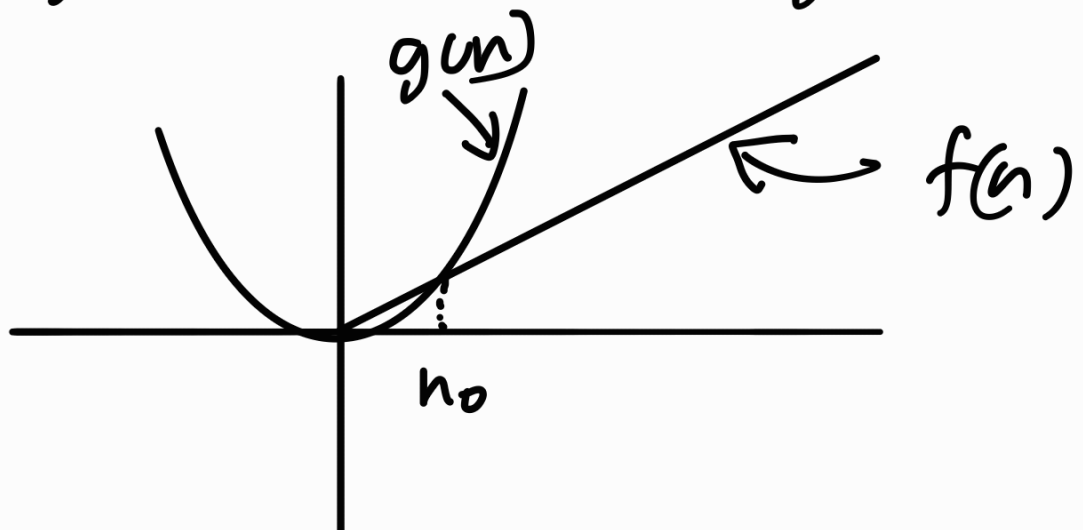
$$f(n) = O(g(n))$$

Ex: $f(n) = 4n$

Claim: $f(n) = O(n^2)$

we must show $c > 0$, $n_0 \geq 0$

$$\forall n \geq n_0 : f(n) \leq c \cdot g(n)$$



Pick $c=1$, $n_0=4$

$$\forall n \geq n_0: 4n \leq 1 \cdot n^2$$

then for $c>0$, $n_0 \geq 0$:

We have

$$\forall n \geq n_0: f(n) \leq c \cdot g(n)$$

$$\therefore f(n) = O(g(n))$$

Pick $c=4$, $n_0=1$

$$\forall n \geq 1: 4n \leq 4 \cdot n^2$$

Another possible picks for
 c and n_0 .

Question: is

$$4n = O(4n^2) \quad \text{yes}$$

$$4n = O(\frac{1}{2}n^2) \quad \text{yes}$$

$$C = 100$$

$$O(4n^2) = O(n^2) = O(\frac{1}{100}n^2)$$

Is $n^3 = O(n^2)$ (no)

I want to prove n^3 is not $O(n^2)$

can we find $c > 0, n_0 \geq 0$

$$\forall n \geq n_0 : f(n) \leq c \cdot g(n)$$

$$\neg [\exists c > 0, n_0 \geq 0 : \forall n \geq n_0 : f(n) \leq c \cdot g(n)]$$

$$\forall c > 0, n_0 \geq 0 : \neg [\forall n \geq n_0 : f(n) \leq c \cdot g(n)]$$

$$[\forall c > 0, n_0 \geq 0 : \exists n \geq n_0 : f(n) > c \cdot g(n)]$$

Let $c > 0$ and $n_0 \geq 0$

$$\text{wts } f(n) > c \cdot g(n)$$

$$n^3 > c \cdot n^2$$

$$n = (c+1)$$

let $n_0 = \max(c, c+1)$

then

$$\forall n \geq n_0: f(n) > c \cdot n^2$$

$\therefore f(n)$ is not $O(n^2)$

Note that

$$f(n) = 4n^3 + 4n + 5$$

$$f(n) = O(n^3) \quad \checkmark$$

$$f(n) = O(n^4) \quad \checkmark$$

$$f(n) = O(n^5) \quad \checkmark$$

However, the tightest $O(\cdot)$ function for $f(n)$ is $O(n^3)$

when we write the Big-O function for $f(n)$ we want to pick the

tightest bound as it is more useful

Common distinct functions

Name	Example $f(n)$	generic $f(n)$	tightest $O(\cdot)$
constant	$f(n) = b$	$f(n) = c$	$O(1)$
log	$f(n) = 2 \log_2 n + 5$	$f(n) = a \log_b n$	$O(\log n)$
linear	$f(n) = 5n + b$	$f(n) = cn$	$O(n)$
$n \log n$	$f(n) = 5n \log_2 n + b$	$f(n) = cn \log_b n$	$O(n \log n)$
quadratic	$f(n) = 5n^2 + n + 2$	$f(n) = an^2$	$O(n^2)$
cubic	$f(n) = 6n^3 + 2n^2 + n + 5$	$f(n) = cn^3$	$O(n^3)$
deg-k polynomial		$f(n) = a_n n^k + a_{n-1} n^{k-1} + \dots + 1$	$O(n^k)$

Exponential $f(n) = n \dots 2^n \quad O(2^n)$
 $f(n) = 3^n \dots 3^n \quad O(3^n)$

functions later on this $\neq O(\text{functions earlier in this list})$
 list

$$2^n \neq O(n^{100})$$

$$3^n \neq O(n)$$

However,

$$\text{functions earlier} = O(\text{functions later})$$

$$n = O(2^n)$$

$$n^{100} = O(2^n)$$

;

Other asymptotic notations

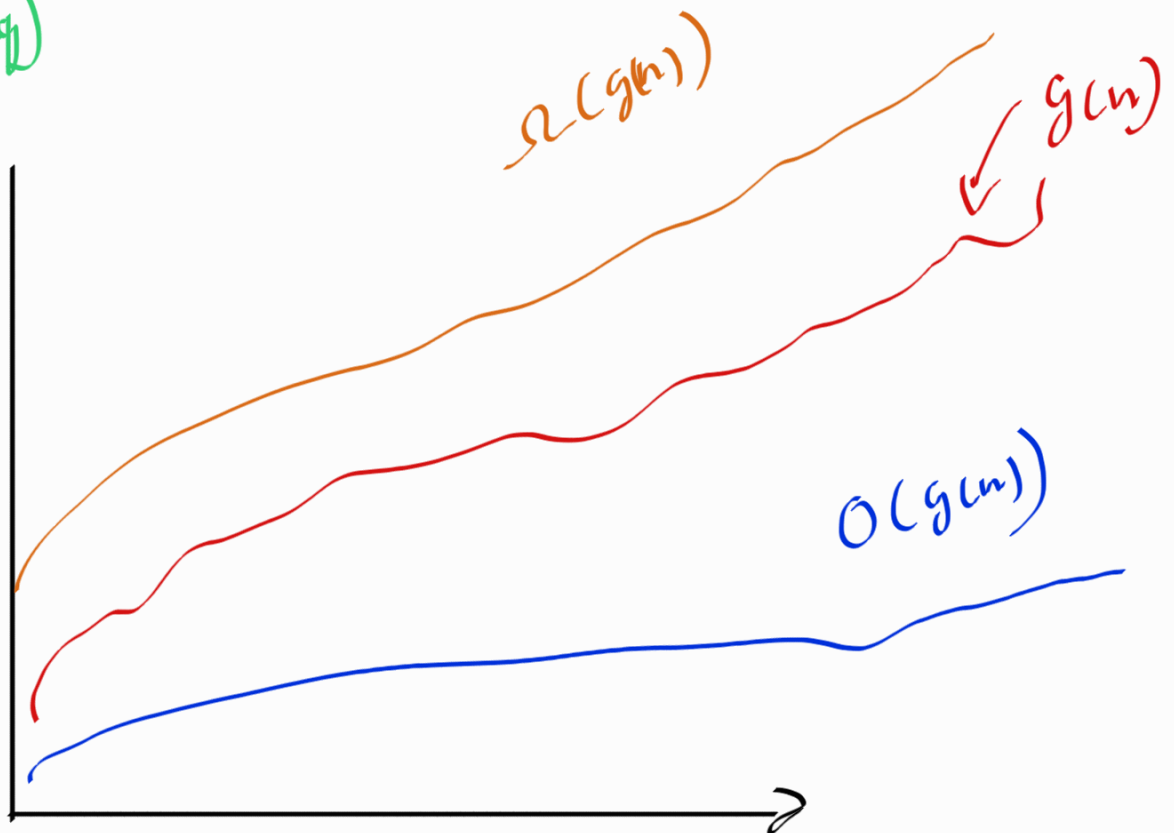
Def Big-Omega (Ω)

Given two functions f and g , we say f grows no slower than g , f is $\Omega(g)$ if

$$\exists d > 0, n_0 > 0: \forall n \geq n_0: f(n) \geq d \cdot g(n)$$

Def Big-theta (Θ)

Given two functions, f and g , we say f is $\Theta(g)$ if $f = O(g)$ and $f = \Omega(g)$



Claim: $f(n) = n^3 + 2n + 5$ is $\Omega(n^3)$

WTS! $\exists d > 0, n_0 \geq 0 : \forall n \geq n_0 : f(n) \geq c \cdot n^3$

Pick $d=1$

$$\forall n \geq 0 : n^3 + 5 > n^3$$

$$\forall n \geq 0 : \underbrace{n^3 + 2n + 5}_{f(n)} \geq n^3 + 5 > \underbrace{n^3}_{g(n)}$$

$$\forall n \geq 0 : f(n) > 1 \cdot g(n)$$

So we picked $d=1, >0$ $n_0=0 \geq 0$

$$\forall n \geq n_0 : f(n) \geq 1 \cdot n^3$$

$$\therefore f(n) = \Omega(n^3)$$

linearSearch($A[1 \dots n], x$):

Input: an array $A[1 \dots n]$ and an element x

Output: is x in the (possibly unsorted) array A ?

```
1 for  $i := 1$  to  $n$ :  
2   if  $A[i] = x$  then  
3     return True  
4 return False
```

(a) Linear Search.

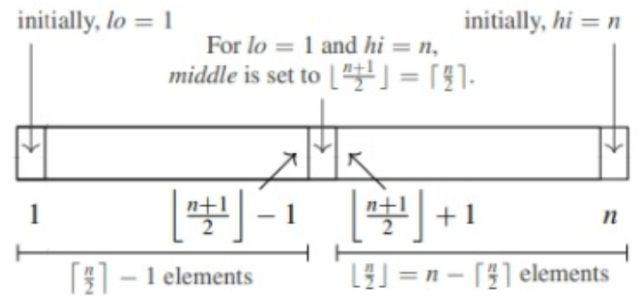
binarySearch($A[1 \dots n], x$):

Input: a sorted array $A[1 \dots n]$; an element x

Output: is x in the (sorted) array A ?

```
1  $lo := 1$ 
2  $hi := n$ 
3 while  $lo \leq hi$ :
4    $middle := \lfloor \frac{lo+hi}{2} \rfloor$ 
5   if  $A[middle] = x$  then
6     return True
7   else if  $A[middle] > x$  then
8      $hi := middle - 1$ 
9   else
10     $lo := middle + 1$ 
11 return False
```

(b) The pseudocode for Binary Search.



(c) An illustration of the first split in binary search.